

Understanding Git: Version Control in Practice

Tianjinwei

2026-07-14

Table of Contents

Introduction	1
1. Where Does Git Come From?	1
2. The Problem of Version Control	1
3. How Does Git Solve It?	1
4. Practical Tips	4
5. Essential Git Commands	5

Introduction

This paper is about Git's theory and practice.

1. Where Does Git Come From?

You have possibly heard that Git descends from Linux. That is right, despite the fact that the first version of Git does not directly originate from it. However, this still makes sense. Git solved many problems when designing the Linux OS with many developers online. That is the problem of version control — the most troublesome problem when working with many developers.

2. The Problem of Version Control

We may fail to retreat to former code when we find our modification was wrong. There are possibly some conflicts when we push our code online while other developers are also modifying.

3. How Does Git Solve It?

This is my personal commit history. Yes, I have two computers committing to this project. However, I forgot that I had committed on another computer, and when I ran `git push` again, the system did not know which one was right. After `git pull`, the conflicts occurred.

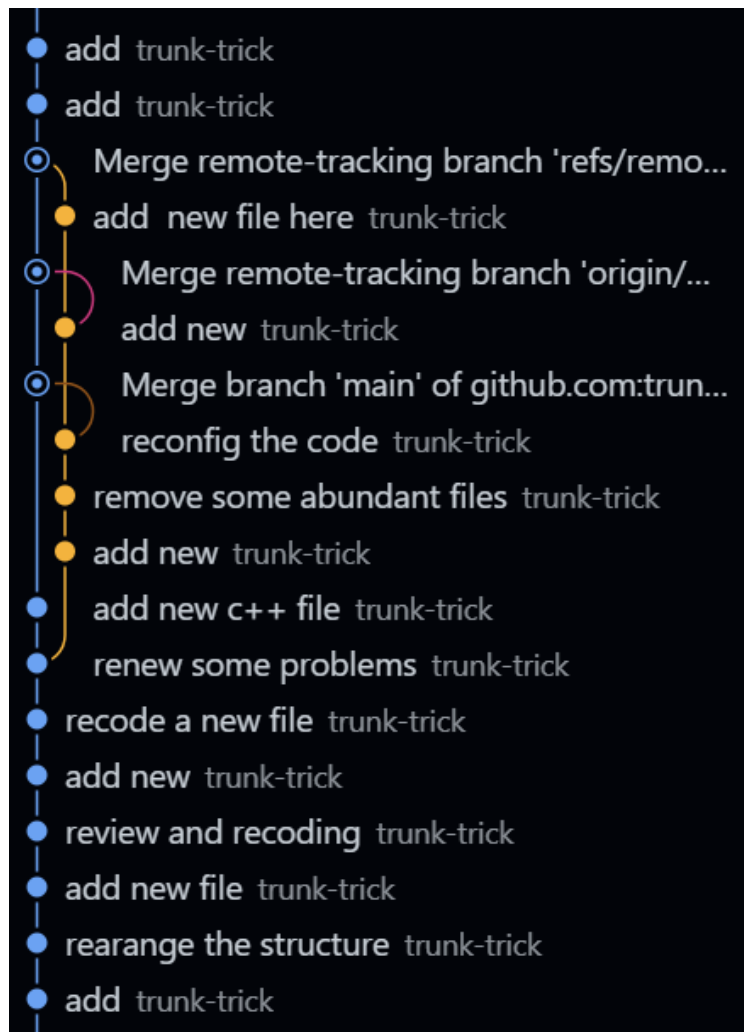


Figure 1: commit history

3.1 Case 1: Conflicts in Different Places

If the conflicts lie in different places (like creating a new file), we do not need to do anything else. We can simply run:

```
git pull
```

That is the combination of `git fetch` and `git merge`.

3.2 Case 2: Conflicts in the Same File

If you just want to keep the local modification (discarding the remote modification), you can solve it with `git rebase`.

If two versions of commits lie in almost the same file (for example, you possibly want to do the same thing here), and it is a joint effort — that is, you want to keep part of the remote modification.

3.3 Resolving Conflicts Step by Step

First, try `git pull` and find that there are conflicts.

If you have not committed your modification:

```
git stash
```

or

```
git commit
```

They actually play the same role here: that is, to remove your local unfinished code so that the remote code can be directly put here.

Get Remote Code

```
git fetch origin main
```

(or even `git pull`)

And now try:

```
git merge origin/main
```

(or `git stash pop`)

If it succeeds, then everything is done.

Now we should look at the conflicts the IDE tells you, and keep what we want to keep. Then everything is done.

3.4 Switching to the Remote Version

If you want to switch to the remote version (discard the local modification):

```
git fetch origin main
git reset --hard origin/main
```

And if you just want some files to be set to the remote version:

```
git fetch origin main
git checkout origin/main -- /path/to/file.cpp
```

3.5 Resolving Conflicts with --theirs

There is actually something used more often:

```
git fetch origin main
git merge origin/main
```

Right after this command, when conflicts occur, we can use:

```
git checkout --theirs path/to/conflict_files.cpp
```

And do not forget to add again:

```
git add path/to/conflict_file.cpp
```

(Actually, you can use `git add .`)

4. Practical Tips

4.1 Changing the Remote URL

What if you find out that your GitHub repo name is too bad for usage, like `---name-_example`? Yes, so you go to github.com to change it. Then what should you do with the local repo related to it?

Use `git remote add <name> <url>` to add it again?

If you do that, the system says:

```
error: remote origin already exists.
```

So actually, you should set the URL again:

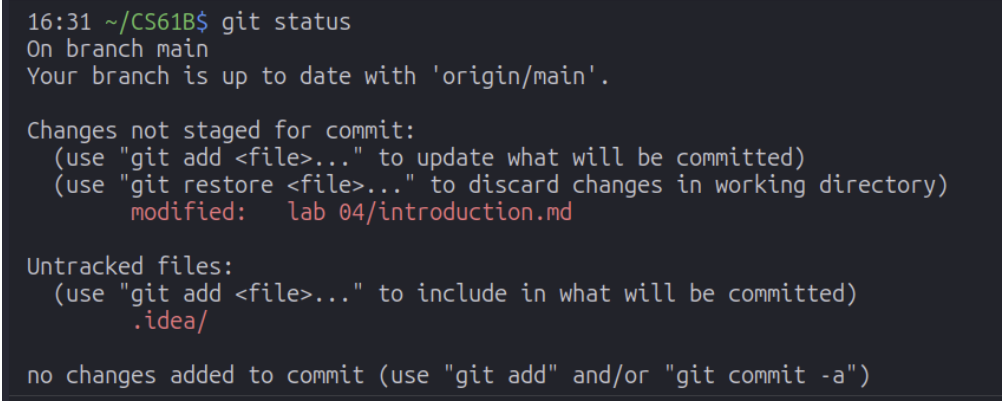
```
git remote set-url origin <repo-url>
```

5. Essential Git Commands

5.1 git status

`git status` is a helpful tool — used for looking up problems.

For example, if I modified one file without commit:



```
16:31 ~/CS61B$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   lab 04/introduction.md

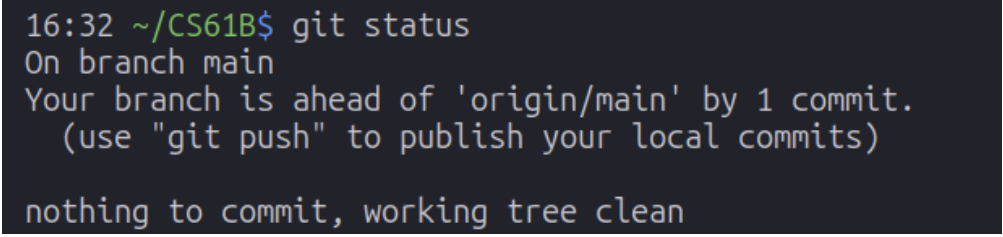
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .idea/

no changes added to commit (use "git add" and/or "git commit -a")
```

Figure 2: git status before commit

Like this: it tells you that you have changes that need to be committed, and which files are not tracked (namely, ignored).

And after you commit, you will see:



```
16:32 ~/CS61B$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

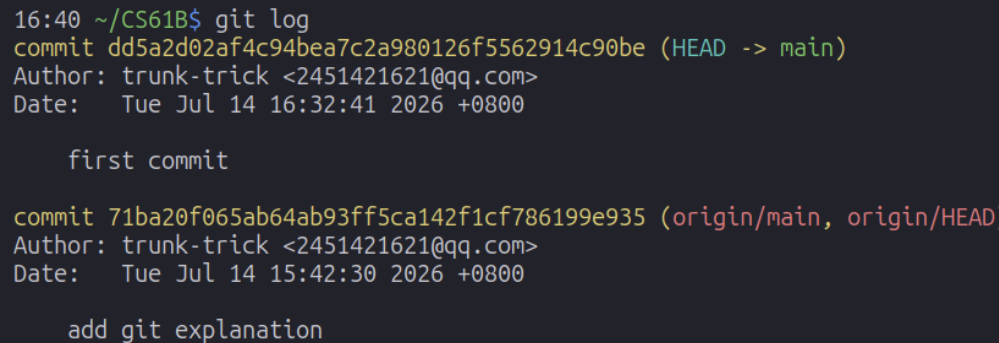
nothing to commit, working tree clean
```

Figure 3: git status after commit

It updates to keep you informed!

5.2 git log

`git log` is used to look at all our commit history!

A terminal window with a dark background showing the output of the 'git log' command. The output lists two commits. The first commit is the 'first commit' with hash 'dd5a2d02af4c94bea7c2a980126f5562914c90be' on the 'main' branch. The second commit is 'add git explanation' with hash '71ba20f065ab64ab93ff5ca142f1cf786199e935' on both 'origin/main' and 'origin/HEAD' branches. Both commits are by 'trunk-trick' and dated 'Tue Jul 14 2026'.

```
16:40 ~/CS61B$ git log
commit dd5a2d02af4c94bea7c2a980126f5562914c90be (HEAD -> main)
Author: trunk-trick <2451421621@qq.com>
Date:   Tue Jul 14 16:32:41 2026 +0800

    first commit

commit 71ba20f065ab64ab93ff5ca142f1cf786199e935 (origin/main, origin/HEAD)
Author: trunk-trick <2451421621@qq.com>
Date:   Tue Jul 14 15:42:30 2026 +0800

    add git explanation
```

Figure 4: git log

Just as you can see, we can look up all the commit history of this branch! Not just local!

5.3 git restore

If you want to restore bad changes to a former version in history, use this command:

```
git restore path/to/file
```

This can only do the default action: restore to the most recent commit.

However, there is indeed a method for us to restore to an earlier version of commits:

```
git restore --source=[commit ID] path/to/file
```

End of document.